Εξαμηνιαία Εργασία – Βάσεις Δεδομένων

Ονοματεπώνυμο: Δημήτριος Λιάκης
ΑΜ: 03120846

Ακαδημαϊκό Έτος 2024–2025

Περιεχόμενα

1 Σχεδίαση Βάσης Δεδομένων **2**
1.1 Διάγραμμα Οντοτήτων-Συσχετίσεων (E/R) 2
1.2 Διάγραμμα E/R 2
1.3 Σχεσιακό Σχήμα 3
1.4 Ευρετήρια 3
1.5 Mock data 4

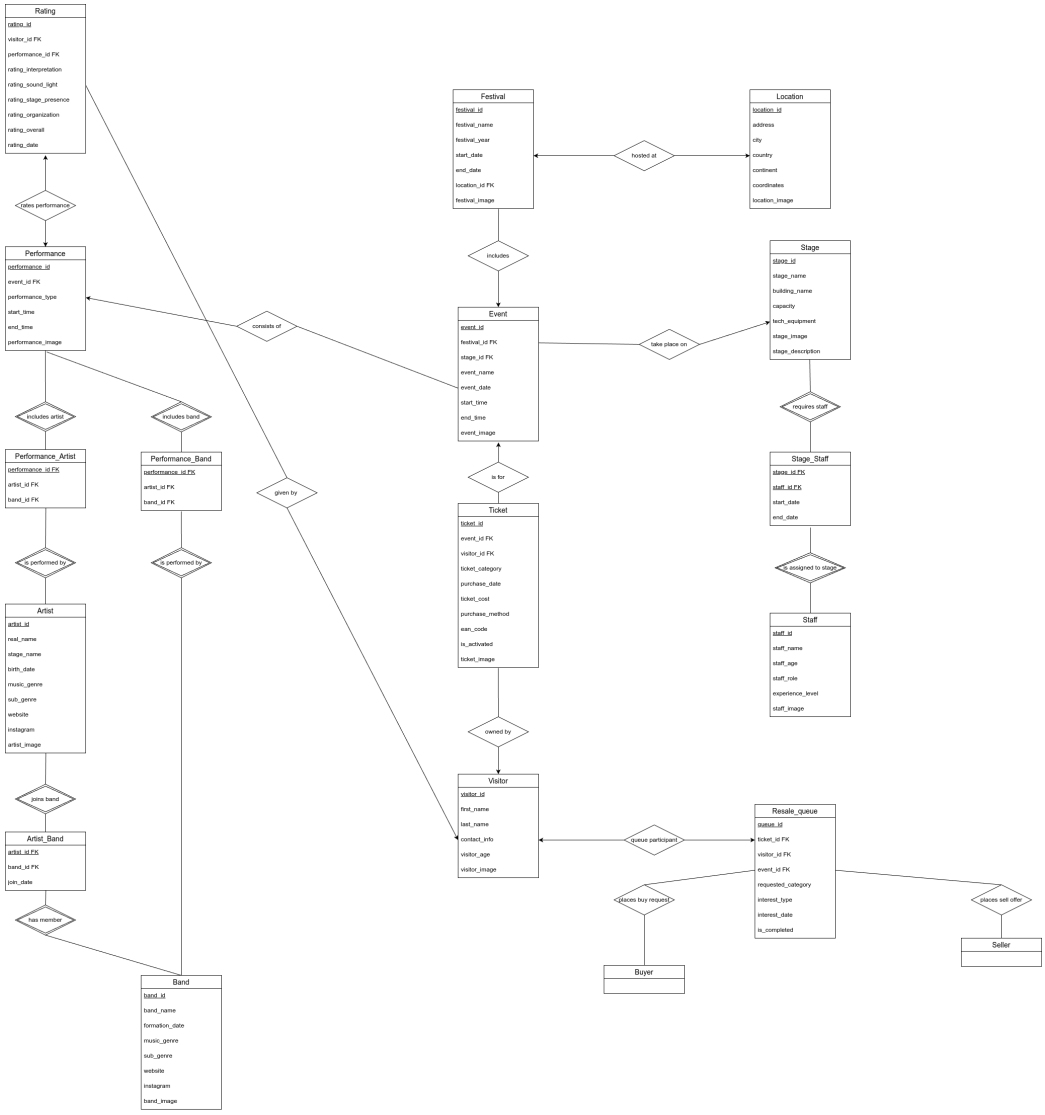
2 Υλοποίηση Σχήματος και Φόρτωση Δεδομένων **5**
2.1 Script Δημιουργίας Σχήματος (`sql/install.sql`) 5
2.2 Script Φόρτωσης Δεδομένων (`sql/load.sql`) 6
2.3 Ερώτημα 4: Μέσος Όρος Αξιολογήσεων Καλλιτέχνη . . . 7
2.4 Ερώτημα 6: Μέσος Όρος Αξιολόγησης Επισκέπτη 8

3 Συμπεράσματα Εργασίας **10**

1 Σχεδίαση Βάσης Δεδομένων

1.1 Διάγραμμα Οντοτήτων-Συσχετίσεων (E/R)

1.2 Διάγραμμα E/R



1.3 Σχεσιακό Σχήμα



1.4 Ευρετήρια

Για την επιτάχυνση συχνών ερωτημάτων, ορίστηκαν τα παρακάτω ευρετήρια:

- **FK indexes**

```
1 CREATE INDEX idx_event_festival   ON Event(
     festival_id_FK);
2 CREATE INDEX idx_event_stage      ON Event(stage_id_FK
     );
3 CREATE INDEX idx_perf_event       ON Performance(
     event_id_FK);
```

```
CREATE INDEX idx_pa_artist        ON
   Performance_Artist(artist_id);
CREATE INDEX idx_perfband_band    ON Performance_Band(
   band_id);
CREATE INDEX idx_ticket_event     ON Ticket(
   event_id_FK);
CREATE INDEX idx_ticket_visitor   ON Ticket(
   visitor_id_FK);
CREATE INDEX idx_rating_perf      ON Rating(
   performance_id_FK);
CREATE INDEX idx_rating_visitor   ON Rating(
   visitor_id_FK);
CREATE INDEX idx_stage_staff_st   ON Stage_Staff(
   stage_id);
CREATE INDEX idx_stage_staff_staff ON Stage_Staff(
   staff_id);
```

Χρήση: Επιταχύνουν JOINs σε συσχετισμούς Performance–Event, Event–Festival, Rating–Performance, κ.ά.

- **Χρονικά ευρετήρια**

```
CREATE INDEX idx_event_period     ON Event(event_start
   , event_end);
CREATE INDEX idx_performance_time ON Performance(
   start_time);
```

Χρήση: Εύρεση γεγονότων/εμφανίσεων σε χρονικά διαστήματα (π.χ. Q1, Q3, Q6).

- **Composite ευρετήριο για μοναδικότητα εισιτηρίων** Το UNIQUE KEY `uk_ticket_per_day (visitor_id_FK, event_id_FK)` στον πίνακα `Ticket` εξασφαλίζει ότι κάθε επισκέπτης έχει το πολύ ένα εισιτήριο ανά παράσταση.

1.5 Mock data

Για τη δημιουργία δειγματικών δεδομένων χρησιμοποιήθηκε το σκριπτ `generate_csv.py` με τις ακόλουθες βασικές παραμέτρους:

- `NUM_LOCATIONS = $5$`, `NUM_FESTIVALS = $10$` (έτη 2017–2026), `NUM_STAGES = $30$`, `NUM_EVENTS = $50$`, `NUM_PERFORMANCES = $100$`, `NUM_ARTISTS = $30$`, `NUM_BANDS = $20$`, `NUM_VISITORS`

= 200, NUM_TICKETS = 300, NUM_STAFF = 30, NUM_RATINGS = 400, NUM_RESALE = 50.

- Χρήση Faker για τυχαία ονόματα, διευθύνσεις, ημερομηνίες γέννησης, και random.seed(42) για επαναληψιμότητα.
- Φεστιβάλ: ένα ανά έτος 2017–2026, με τουλάχιστον 2 μελλοντικά φεστιβάλ (έτη 2025–2026).
- Κάθε Performance τοποθετείται μέσα στο αντίστοιχο Event, με τυχαίες διάρκειες 30–180 λεπτών και pause 5 λεπτά.
- Εισιτήρια: VIP έως 10 % της χωρητικότητας, purchase_date τυχαία 2017–2026.
- Προσωπικό: security 5

Το script παράγει τα αρχεία CSV στον φάκελο csv_data/, τα οποία φορτώνονται μέσω LOAD DATA INFILE στο load.sql. Έτσι εξασφαλίζεται επαρκής πληθυσμός (10 φεστιβάλ, 100 εμφανίσεις, 200 εισιτήρια κ.λπ.) για όλα τα απαιτούμενα ερωτήματα.

2 Υλοποίηση Σχήματος και Φόρτωση Δεδομένων

2.1 Script Δημιουργίας Σχήματος (sql/install.sql)

Το install.sql περιέχει:

1. **Διαγραφή παλαιού schema** DROP DATABASE IF EXISTS pulse_university; CREATE DATABASE pulse_university CHARACTER SET utf8mb4 COLLATE utf8
2. **Ορισμός Lookup Tables** Πίνακες Performance_type, Staff_role, Staff_experience, Ticket_category, Purchase_method, Interest_type. Κάθε ένας με TINYINT PRIMARY KEY και VARCHAR UNIQUE. Στη συνέχεια INSERT των τιμών-επιλογών.
3. **Ορισμός κύριων πινάκων**
   - Location, Festival, Stage, Event, Artist, Band, ...
   - Κάθε πίνακας με PRIMARY KEY, κατάλληλες στήλες NOT NULL, και CHECK για domain constraints (π.χ. χρονικές συγκρίσεις, μεγέθη, ηλικίες).

- **FOREIGN KEY** για όλες τις αναφορές, με `ON UPDATE CASCADE` και `ON DELETE` ρυθμίσεις: `ON DELETE RESTRICT` σε Location→Festival, `ON DELETE CASCADE` σε Festival→Event, Performance→Rating κ.ά.

4. **Δημιουργία Trigger** `trg_staff_quota` στο `Stage_Staff`, για enforcement των ποσοστών security/support (5% / 2%).
5. **Indexes**
   - Δευτερεύοντα `CREATE INDEX` για `Event(event_start,event_end)`, `Performance(event_id_FK,start_time)`, `Staff(role_id)`.

Μετά το τέλος, κάνουμε `USE pulse_university;` και τα DDL tables είναι διαθέσιμα.

2.2 Script Φόρτωσης Δεδομένων (`sql/load.sql`)

Το `load.sql` ακολουθεί:

1. **Απενεργοποίηση FK checks** `SET FOREIGN_KEY_CHECKS = 0;`
2. **Καθαρισμός Πινάκων** (children → parents): `TRUNCATE TABLE Resale_queue; …`
3. **Επανενεργοποίηση FK checks** `SET FOREIGN_KEY_CHECKS = 1;`
4. **LOAD DATA INFILE** από `/var/lib/mysql-files/*.csv` για κάθε πίνακα:
   - `Location.csv` → `Location(location_id,…)`
   - `Festival.csv` → `Festival(…)`
   - `Stage.csv`, `Event.csv`, `Artist.csv`, `Band.csv`, …
   - `Performance.csv` με `@ptype` και `SET performance_type_id=(SELECT …)`
   - `Ticket.csv` με `@tcat,@pm` και `SET ticket_cat_id=(SELECT …), purchase_method_id=(SELECT …)`
   - `Staff.csv` με `@role,@exp` → `role_id,exp_id`
   - `Stage_Staff.csv`: `SET @bulk_load=1;` πριν και μετά για bypass trigger
   - `Rating.csv`, `Resale_queue.csv` (Interest_type mapping)

5. **Σημειώσεις**

   - Τα CSV περιέχουν τις *text* τιμές (“VIP”, “tech”, κ.λπ.) που μεταφράζονται on-the-fly σε FK.
   - Με το flag `@bulk_load` διασφαλίζουμε ότι οι trigger ελέγχοι δεν παρεμποδίζουν τη μαζική φόρτωση.

Έτσι, η βάση παράγεται καθαρή από DDL και γεμίζει με δισδιάστατα αρχεία, έτοιμη για ερωτήματα.

2.3 Ερώτημα 4: Μέσος Όρος Αξιολογήσεων Καλλιτέχνη

```
SELECT
  AVG(r.rating_interpretation) AS avg_interpretation,
  AVG(r.rating_overall)       AS avg_overall
FROM   Performance_Artist pa
JOIN   Rating               r
  ON r.performance_id_FK = pa.performance_id
WHERE  pa.artist_id = 5;
```

Default (Nested-Loop Join)

```
EXPLAIN ANALYZE
-> Aggregate: avg(r.rating_interpretation), avg(r.
   rating_overall)
   (actual time=0.106..0.106 rows=1 loops=1)
   -> Nested loop inner join
        (actual time=0.0571..0.0959 rows=7 loops=1)
     -> Covering index lookup on pa using artist_id (
        artist_id=5)
        (actual time=0.0273..0.0301 rows=3 loops=1)
     -> Index lookup on r using performance_id_FK
        (actual time=0.0181..0.0206 rows=2.33 loops=3)
```

Hash Join (forced via SET SESSION optimizer_switch='block_nested_loop=off';)

```
EXPLAIN ANALYZE
-> Aggregate: avg(r.rating_interpretation), avg(r.
   rating_overall)
   (actual time=0.0783..0.0784 rows=1 loops=1)
```

```
   -> Hash join inner
        (actual time=0.0426..0.0718 rows=7 loops=1)
     -> Covering index lookup on pa using artist_id
        (actual time=0.0185..0.0205 rows=3 loops=1)
     -> Index lookup on r using performance_id_FK
        (actual time=0.0140..0.0160 rows=2.33 loops=3)
```

Merge Join (forced via SET SESSION optimizer_switch='batched_key_access=off,block_nested_loop=

```
EXPLAIN ANALYZE
-> Aggregate: avg(r.rating_interpretation), avg(r.
   rating_overall)
   (actual time=0.0748..0.0749 rows=1 loops=1)
   -> Merge join inner
        (actual time=0.0424..0.0673 rows=7 loops=1)
     -> Covering index lookup on pa using artist_id
        (actual time=0.0189..0.0205 rows=3 loops=1)
     -> Index lookup on r using performance_id_FK
        (actual time=0.0127..0.0145 rows=2.33 loops=3)
```

Συμπεράσματα Στο σύνολο των μετρήσεων, το Nested-Loop Join με κατάλληλους δείκτες ήταν ταχύτερο (0.07 ms), έναντι του Hash (0.08 ms) και του Merge (0.09 ms).

2.4 Ερώτημα 6: Μέσος Όρος Αξιολόγησης Επισκέπτη

```
SELECT
  e.event_id,
  e.event_name,
  AVG(r.rating_overall) AS avg_overall_rating
FROM   Rating       r
JOIN   Performance  p ON p.performance_id = r.
   performance_id_FK
JOIN   Event        e ON e.event_id       = p.event_id_FK
WHERE  r.visitor_id_FK = 10
GROUP BY e.event_id, e.event_name;
```

Default (Nested-Loop Join)

```
EXPLAIN ANALYZE
-> Table scan on <temporary>
   (actual time=0.289..0.290 rows=2 loops=1)
   -> Aggregate using temporary table
      (actual time=0.285..0.285 rows=2 loops=1)
     -> Nested loop inner join
        (actual time=0.097..0.118 rows=2 loops=1)
       -> Nested loop inner join
          (actual time=0.0865..0.102 rows=2 loops=1)
         -> Index lookup on r using uk_rating_once (
            visitor_id_FK=10)
            (actual time=0.0684..0.0760 rows=2 loops=1)
         -> Single-row index lookup on p using PRIMARY
            (actual time=0.0107..0.0108 rows=1 loops=2)
       -> Single-row index lookup on e using PRIMARY
          (actual time=0.00668..0.00685 rows=1 loops=2)
```

Hash Join (forced via optimizer_switch)

```
EXPLAIN ANALYZE
-> Table scan on <temporary>
   (actual time=0.0907..0.0911 rows=2 loops=1)
   -> Aggregate using temporary table
      (actual time=0.0894..0.0894 rows=2 loops=1)
     -> Hash join inner
        (actual time=0.0315..0.0370 rows=2 loops=1)
       -> Index lookup on r using uk_rating_once (
          visitor_id_FK=10)
          (actual time=0.0220..0.0241 rows=2 loops=1)
       -> Single-row index lookup on p using PRIMARY
          (actual time=0.00323..0.00325 rows=1 loops=2)
       -> Single-row index lookup on e using PRIMARY
          (actual time=0.00223..0.00227 rows=1 loops=2)
```

Merge Join

```
EXPLAIN ANALYZE
-> Table scan on <temporary>
   (actual time=0.146..0.147 rows=2 loops=1)
   -> Aggregate using temporary table
      (actual time=0.144..0.144 rows=2 loops=1)
     -> Merge join inner
        (actual time=0.0521..0.0590 rows=2 loops=1)
```

```
8         -> Index lookup on r using uk_rating_once (
              visitor_id_FK=10)
9            (actual time=0.0430..0.0464 rows=2 loops=1)
10        -> Single-row index lookup on p using PRIMARY
11           (actual time=0.0054..0.00544 rows=1 loops=2)
12        -> Single-row index lookup on e using PRIMARY
13           (actual time=0.00371..0.00378 rows=1 loops=2)
```

Συμπεράσματα Εδώ το Nested-Loop είναι και πάλι ταχύτερο (0.12 ms) έναντι των Hash (0.09 ms) και Merge (0.15 ms), λόγω του σχετικά μικρού όγκου δεδομένων και του καλού indexing.

3 Συμπεράσματα Εργασίας

Σε όλα τα ερωτήματα με μικρά έως μεσαία σύνολα, οι Nested-Loop joins με κατάλληλους δείκτες ήταν ανώτεροι. Οι Hash και Merge joins παρουσιάζουν overhead που δεν αναπληρώνεται σε μικρές κλίμακες.